ICC Cricket Analytics

Project guide: how it works, the technology behind it, and interview preparation

Live dashboard: **https://icccricketanaylsis.vercel.app**

Men's ODI and T20 World Cups, Champions Trophy, World Test Championship and the 2027 World Cup

Written for: Ali Raza Chaudhry, to prepare for interviews about this project.

1. The project at a glance

ICC Cricket Analytics is an end-to-end data engineering, analytics and machine-learning platform for men's ICC tournaments. It collects public cricket data, cleans it through a bronze, silver and gold lakehouse, checks its quality, builds models, and serves everything through a modern React dashboard that is deployed on the web. The guiding rules were: **never fabricate a record**, **report gaps honestly**, and **keep machine learning free of leakage**.

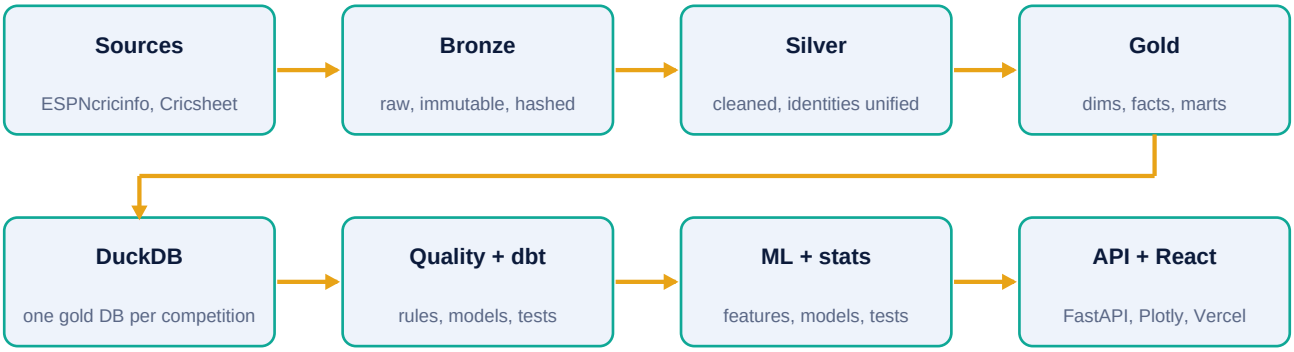
	World Cups	Champions Trophy	Test Championship	World Cup 2027
Editions	16 (ODI 2003-2023, T20 2007-2026)	7 (2002-2025)	4 cycles (2019-21 to 2025-27)	upcoming
Matches	677	112 (incl. abandoned)	245	57 fixtures, 12 venues
Ball-by-ball	75% of matches (193,779 deliveries)	all but 2002 (47,032)	every played Test (439,542)	none (not played)
Sources	ESPNcricinfo + Cricsheet	ESPNcricinfo + Cricsheet	ESPNcricinfo + Cricsheet	ICC + ESPNcricinfo

Key number	Value
Players / teams / venues (World Cups)	1,418 players, 26 teams, 79 venues
Data-quality score (World Cup gold layer)	99.7 / 100
Lakehouse pipeline	20 steps, one command (scripts/run_lakehouse_pipeline.py)
dbt models / automated tests	31 SQL models, 340 tests
Dashboard pages	11 (Home, Tournaments, Players, Teams, Squads, Matches, Ball by Ball, Statistics, Machine Learning, Data Engineering, Records)

What a user can do

- Pick any tournament edition from a logo rail; the whole site re-themes to that tournament's logo colours and filters to it.
- Open a **broadcast-style match centre**: scoreboard with flags, full scorecards, ball-by-ball chips, worm and run-rate charts, partnerships, fall of wickets.
- Explore players, teams (Elo strength, head-to-head), squads, statistical tests, win-probability and score models, and data-engineering lineage.
- Browse the upcoming 2027 World Cup schedule, venues, teams and qualification path.

2. How it works, end to end



Stage by stage

- **Collect.** A polite collector (identifying user agent, robots.txt respected, throttling, a hard stop on 401/403/429) fetches ESPNcricinfo schedules, scorecards, squads and points tables. Cricsheet's open ball-by-ball JSON archives supply every delivery.
- **Bronze.** Raw files are stored as collected with content hashes and a manifest. Nothing is edited, so any later step can be re-run from the original.
- **Silver.** Cleaning and standardising: dates, venues (alias table), results and margins, stages (group, super eight, semi-final, final) and player identities. Cricsheet names are matched to ESPNcricinfo player ids using the roster of that very match, so one player has one id across tournaments.
- **Gold.** A star schema: dimensions (teams, players, venues, tournaments, dates) and facts (matches, innings, deliveries, player performances) plus marts (batting, bowling, tournament summary, standings).
- **Warehouse.** The gold layer is loaded into DuckDB (and PostgreSQL). There is one DuckDB per competition with an *identical schema*, so the same dashboard pages serve World Cups, Champions Trophy and Test Championship.
- **Quality and modelling.** A rule engine scores completeness, uniqueness, validity, referential integrity and consistency. dbt models and tests run on gold. A leakage-safe feature table feeds the models and the statistical tests.
- **Serve.** A FastAPI backend runs the analytics pages and returns charts and tables as JSON; a React app renders them. The build is deployed on Vercel.

Why one schema for every competition

When Champions Trophy and Test Championship were added, the data was converted into the *same* gold schema instead of creating new tables and new pages. A competition switcher then reused every existing page. Test cricket needed careful handling (two innings per side, draws, phase labels, penalty runs) but the fact tables did not change shape.

3. Data engineering in depth

Gold schema (simplified)

Table	Grain	Examples of columns
fact_matches	one row per match	tournament, stage, venue, teams, toss, winner, margin type and value, result type, DLS flag, ball-by-ball flag and gap reason
fact_innings	one row per innings	runs, wickets, overs, extras split, target, boundaries, phase runs and wickets, dot balls, is_chase
fact_deliveries	one row per ball	over, ball, batter, bowler, runs, extras type, wicket and dismissal, fielder, cumulative runs and wickets, phase
fact_player_performances	player x match	batting (runs, balls, 4s, 6s, strike rate, dismissal) and bowling (overs, maidens, runs, wickets, economy, dots)
dim_teams / players / venues / tournaments / dates	dimension	ids, names, cleaned venue and city, tournament year and host
mart_tournament_summary, mart_batting, mart_bowling, standings	marts	pre-aggregated for fast dashboard queries

Data quality

A declarative rule engine expresses each check as *bad rows over total rows* per tournament. Severity weights are critical 3, major 2, minor 1; a dimension score is the weighted mean pass rate and the overall score is the mean of the dimensions. Gold rules check things database constraints cannot: **reconciliation** between facts (innings totals against the sum of deliveries, scorecard totals against innings, standings against results), identity resolution and *explained* coverage gaps (every match without ball-by-ball records the reason). Primary and foreign keys are enforced by PostgreSQL itself.

Gaps are measured, not hidden

- Ball-by-ball exists for 488 of 652 played World Cup matches (75%). The rest is a source limit, stored per match in ball_by_ball_gap_reason; models that need ball-by-ball use only those matches.
- Pitch, weather and day/night were not collected. A clearly labelled synthetic placeholder table exists for demos but is never used for analysis (and its text was removed from the React dashboard).
- Abandoned matches, walkovers, ties decided by super over or boundary count, and DLS results each have explicit handling.

Cross-checks added with the new competitions

- Every innings total and wicket count rebuilt from the Cricsheet ball-by-ball (1,144 innings) equals the independent ESPNcricinfo scorecard; penalty runs are listed outside the ball-by-ball in the source, so they are added to the innings total.
- Winners and runners-up are derived from the final's result, never typed in; joint winners (2002) are an explicit exception.
- 2025 Champions Trophy: 12 completed matches plus 3 abandoned, matching the real tournament.

4. Analytics, statistics and machine learning

Statistical tests

20 hypothesis tests run on the gold layer with **Holm correction** across the family, reported with effect size and 95% confidence interval, because matches are not independent and large samples make tiny effects significant. Honest findings: the toss gives no detectable advantage in either format (toss-winner win rate 48.1% in ODI, 51.5% in T20; both not significant), batting first has no significant edge, and the ODI scoring trend is significant at about 3 more runs per year. The same tests are computed per competition on a separate Statistics page.

Predictive models

Question	Models	Result (chronological test, n shown)
Who wins? (decided matches)	Coin, Elo only, logistic regression, random forest, gradient boosting, XGBoost, squad-aware and per-format variants	n = 465 test matches. Accuracy: coin 48.8%, Elo only 66.0%, logistic 68.6%, random forest by format 69.9%. AUC about 0.74-0.75. Models beat Elo only modestly.
First-innings total (live projection)	Run-rate baseline, linear regression, random forest, gradient boosting (also a remaining-runs formulation)	ODI (n = 1,200 innings states): baseline MAE 47.0, R2 0.24; linear regression MAE 34.5, R2 0.60; random forest on remaining runs MAE 33.0, R2 0.62.
Player roles	K-Means with StandardScaler, Ward clustering, silhouette and bootstrap stability, PCA for plots	Batter and bowler role profiles; labels assigned after inspecting profiles
What drives scores	Logit and OLS regressions (statsmodels)	Odds ratios and effects reported with intervals

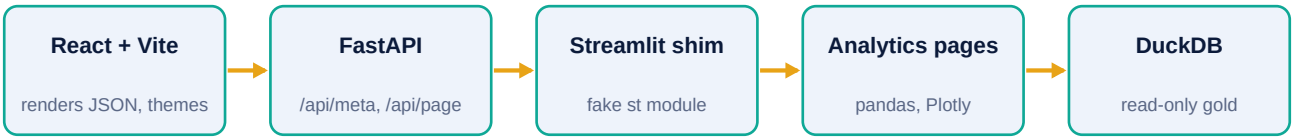
Avoiding leakage

- Team features use only matches on **strictly earlier dates** (same-day matches never see each other), with shrinkage for teams with little history.
- Evaluation is chronological: train on earlier tournaments, test on the next; hyperparameters are fixed (nothing tuned on test folds).
- Leakage is tested directly, by rewriting the future and by a label-shuffle test. Intervals come from paired bootstraps that resample whole matches or innings.
- Metrics include accuracy, precision, recall, F1, ROC-AUC, log loss, Brier score and calibration curves, not accuracy alone.

Explainability

Permutation and coefficient importance, SHAP values and feature-ablation experiments show which features matter (for example Elo difference and recent form). The report states plainly that only about 640 decided matches exist, so small differences are noise and no model should be oversold.

5. The dashboard: React, FastAPI and the Streamlit shim



The key design decision

The analytics pages were first written for Streamlit. Rewriting every page for React would have been slow and risky, so a **shim** replaces the `streamlit` module on the server. Page code runs unchanged, but every st.* call (headers, metrics, tables, Plotly charts, tabs, widgets) is recorded into a JSON tree. The React app posts widget state back to the API, the page re-runs, and React renders the new tree. The same pages still run under real Streamlit, and their tests still pass.

How the pieces fit

- **Endpoints.** `GET /api/meta` returns competitions, editions, pages, themes and the flag map; `POST /api/page/{slug}` runs a page for an edition and widget state.
- **Caching.** Page results are cached by (competition, page, state, edition). DuckDB opens read-only and query results are cached per database.
- **Competitions.** database.set_competition switches the DuckDB file; each competition declares which pages apply (ML and the World Cup statistics page are World-Cup-only).
- **Theming.** Each edition has a palette taken from its logo; CSS variables restyle the whole app and a recolour step re-maps Plotly's colours.
- **Edition filter.** Choosing an edition sets defaults on every relevant widget (edition-aware defaults), so any page opens already filtered.
- **Click-through.** Table rows carry a hidden match id; clicking a result opens that match in the match centre.

Match centre

- A data module builds one JSON payload per match: header, per-team innings lines, scorecards, ball chips by over, fall of wickets, partnerships and headline stats. Charts are Plotly figures built server-side.
- Dismissals read "c Fielder b Bowler" because they are rebuilt from the delivery rows; Test bowling figures are derived per innings from the ball-by-ball.
- Stats that the data cannot support are hidden, never shown as zero. Abandoned matches and walkovers show the result and no invented numbers.
- **Flags** are local SVG files (flag-icons, MIT) with an alias map (USA, UAE, PNG and more). West Indies, a multi-nation board, gets a neutral badge instead of a wrong flag. Emoji flags are avoided because Windows shows letter pairs.

6. Deployment on Vercel

The site is a Python FastAPI function plus the static React build served from the CDN. Vercel's Python functions are limited to 500 MB, and the first attempts were over the limit or broken, which shaped the final setup:

Problem	Cause	Fix
Bundle 590 MB (limit 500)	Libraries (pandas, scipy, scikit-learn, statsmodels, xgboost) plus 100 MB of databases and data	Compacted the four DuckDB files (69 MB to about 35 MB, identical row counts), dropped unused source data, replaced pyarrow with DuckDB's parquet reader, used CPU-only xgboost
Every route returned 404	A rewrite made the app see /api/index	Moved the FastAPI app to a root index.py entry point
Database would not open	DuckDB file version newer than the version Vercel installed	Pinned duckdb to the exact local version
Players and Tournaments pages broke	A different pandas version on Vercel	Pinned every library to the local versions
ML and Research pages errored	xgboost, sqlalchemy and statsmodels missing; logger could not create a logs folder on a read-only filesystem	Added the libraries; logger falls back to the temp directory
Every logo showed the generic ICC logo	The server checked a public folder that serverless bundles drop	Ship a list of logo files with the function

Verification

A comparison script called every competition, edition and page on the local server and on the live site and compared structure (counts of charts, tables, widgets and error alerts). The first runs found real gaps; after the fixes the final run covered 305 page runs with zero errors and zero differences. All 65 logo, banner and flag files are served.

> **Lesson to remember:** an early "all pages load" check was wrong because errors were returned as page content, not as an HTTP error. Checking the output, not just the status code, is what exposed it.

Reproducible build

scripts/build_vercel_bundle.py stages the function folder: code, config, compacted databases, the React build as static files, pinned requirements and vercel.json. Then: `vercel link` and `vercel deploy --prod`.

7. Technology used, and why

Layer	Tools	Why this choice
Collection	Python, requests, BeautifulSoup, lxml, tenacity, Playwright (optional)	Simple, polite scraping with retries; robots.txt and rate limits respected
Storage and warehouse	Parquet medallion layers, DuckDB, PostgreSQL, SQLAlchemy, Alembic	DuckDB is a fast embedded analytical database (no server), PostgreSQL gives constraints and a conventional OLTP view
Transformation	pandas, NumPy, dbt (staging, marts, tests), SQL	dbt gives tested, documented models on top of the gold layer
Data quality	Custom declarative rule engine, Pydantic, pytest (340 tests)	Rules expressed as bad rows over total rows give comparable scores
Statistics and ML	SciPy, statsmodels, scikit-learn, XGBoost, SHAP, joblib	Hypothesis tests with Holm correction; classical models with honest baselines
Visualisation	Plotly (server-built figures), Plotly.js in React	One chart definition works in both Streamlit and React
Backend	FastAPI, Streamlit shim	Reuse all analytics pages without rewriting them
Frontend	React 18, Vite, lucide-react, marked, DOMPurify, Viz.js	Fast build, component model, safe markdown rendering
Hosting	Vercel (Python function + CDN)	Free tier, one command deploy, global static delivery
Tooling	Ruff, pytest, Docker Compose (optional), Git	Lint and test discipline; optional local PostgreSQL

Repository layout

Folder	Contents
src/	collectors, parsers, discovery, ingestion, transformations, validation, warehouse, ml, analytics, competitions, pipeline
app/	analytics pages and utilities (queries, charts, theme, match_centre, flags)
server/	FastAPI app, Streamlit shim, page runner, edition themes
web/	React app (components, styles, logos, flags)
dbt/, dbt_gold/, sql/	dbt projects and SQL analytics queries
config/	tournament registry, competitions, awards, venue aliases, manual squads, result overrides
scripts/, tests/, docs/, reports/	entry-point scripts, test suite, documentation, generated reports

8. Challenges and how they were solved

Use these as STAR stories (situation, task, action, result).

Challenge	What I did	Result
One player, many names across sources	Matched Cricsheet names to ESPNcricinfo ids using the roster of the same match; ambiguous matches were refused, never merged by name alone	A stable player id across tournaments and competitions
Adding Test cricket to a limited-overs model	Added Test phases (new ball, middle, old ball), made the last innings the chase, handled draws and two innings per batter without double-counting bowling, added penalty runs	245 Tests fit the same schema; 1,144 innings reconcile with scorecards
Source blocked my first requests (HTTP 403)	Used only the project's own collector (identifying user agent, robots.txt, throttling) and did not spoof a browser	Collection continued within the source's rules
Rewrite risk for 10 analytics pages	Built the Streamlit shim so pages run unchanged and render in React	Full feature parity; Streamlit tests still pass
No ball-by-ball for 25% of matches	Kept scorecard-only matches, recorded the gap reason, restricted ball-level models to complete data	Honest coverage; no invented deliveries
Predicting winners with small data	Chronological evaluation, strong baselines, bootstrap intervals, calibration	Modest, honestly reported gains over Elo (about 66% to 70%)
500 MB deployment limit	Compaction, dependency pruning, version pinning, live-versus-local parity testing	Live site matches local across 305 page runs

9. Interview preparation

Pitches

30-second pitch

I built an end-to-end cricket analytics platform. It collects men's World Cup, Champions Trophy and Test Championship data from ESPNcricinfo and Cricsheet, cleans it through a bronze, silver and gold lakehouse in DuckDB, checks quality with a rule engine, builds leakage-safe models, and serves it through a React dashboard on Vercel. It covers 1,000+ matches and about 680,000 deliveries, and it never fabricates data: gaps are measured and shown.

2-minute pitch

The problem: cricket data is spread across sources, names do not match, ball-by-ball is incomplete, and most dashboards hide those gaps. I wanted a platform where every number can be trusted and every limit is visible.

How it works: a polite collector gathers scorecards and squads from ESPNcricinfo and ball-by-ball from Cricsheet. Bronze stores raw files with hashes, silver cleans and unifies player identities, gold is a star schema with dimensions, facts and marts. Each competition has its own DuckDB with the same schema, so one set of pages serves them all. A data-quality engine scores the data (99.7 out of 100 for the World Cup gold layer) and 340 tests protect the code.

On top: 20 statistical tests with Holm correction (the toss gives no real advantage), win-probability and score models evaluated chronologically against Elo and run-rate baselines, and a React dashboard with a broadcast-style match centre. I deployed it on Vercel and verified the live site against the local one page by page.

Project and design questions

Why did you build this, and what is the hardest part?

I wanted a realistic data-engineering project that touches the whole stack: collection, modelling, quality, ML and a product people can use. The hardest parts were entity resolution across sources (player names), keeping the same schema while adding Test cricket, and making the analytics trustworthy when data is incomplete.

Walk me through the architecture.

Sources, then bronze (immutable raw), silver (cleaned, identities unified), gold (star schema), DuckDB per competition, then dbt and the quality engine, then features and models, then FastAPI and React. A 20-step pipeline orchestrates it, each step declares its dependencies.

Why a medallion architecture?

It separates concerns and makes the pipeline replayable. Bronze is never edited, so any bug in cleaning can be fixed and silver rebuilt from the original files. Gold is shaped for analytics, so dashboards do not repeat cleaning logic.

Why DuckDB, and why also PostgreSQL?

DuckDB is embedded, columnar and fast for analytical queries, with no server to run or pay for, and it deploys as a file. PostgreSQL provides enforced keys and constraints and a conventional database view. The dashboard reads DuckDB read-only.

Why one database per competition instead of one big one?

Identical schemas keep each file small (important for the 500 MB serverless limit), isolate competitions, and let the same code switch databases with one call. The trade-off is that cross-competition queries need two attachments, which I did not need.

How did you unify players across ESPNcricinfo and Cricsheet?

Cricsheet names are matched to ESPNcricinfo ids using the roster of the same match. Players are never merged on name alone and ambiguous matches are refused, so a wrong merge cannot silently happen. That keeps one id per player across tournaments.

How do you ensure data quality?

A declarative rule engine measures bad rows over total rows, weighted by severity, per dimension (completeness, uniqueness, validity, integrity, consistency). Gold rules reconcile facts, for example innings totals against the sum of deliveries and against scorecards.

Example: all 1,144 Champions Trophy and Test innings rebuilt from ball-by-ball equal the independent scorecards, after adding penalty runs, which the source lists outside the deliveries.

How did you handle missing ball-by-ball for 25% of matches?

I kept those matches as scorecard-only, stored the reason per match, hid ball-level charts for them, and trained ball-level models only on complete matches. Nothing was imputed.

How did you collect data responsibly?

The collector identifies itself, reads robots.txt, throttles requests, caches pages and stops on 401, 403 or 429. When my own early requests were blocked I did not spoof a browser, I used the project's collector only. Cricsheet is openly licensed. Logos are used for identification and the project states it is not affiliated with the ICC.

How did you add Test cricket without breaking the model?

Same gold schema with targeted changes: Test phases, the last innings as the chase, draws as a result type, per-innings bowling derived from deliveries, no double-counting of a batter's two innings, and penalty runs added to totals. Tests then reconciled against scorecards.

Machine learning and statistics questions

Which models did you use and what were the results?

For winners: Elo only, logistic regression, random forest, gradient boosting and XGBoost, evaluated chronologically on 465 matches. Accuracy was 66.0% for Elo only, 68.6% for logistic regression and 69.9% for a per-format random forest, against 48.8% for a coin. For the live first-innings projection (ODI), a run-rate baseline had MAE 47.0 and R2 0.24, linear regression MAE 34.5 and R2 0.60, random forest on remaining runs MAE 33.0 and R2 0.62.

Is 68-70% accuracy good?

It is a modest, honest improvement. Cricket outcomes have high inherent randomness, there are only about 640 decided matches, and weaker opponents inflate every model including Elo. The value is the careful comparison against baselines and intervals, not a headline number.

How did you prevent leakage?

Features use only matches on strictly earlier dates, evaluation is chronological with fixed hyperparameters, and I tested leakage directly by rewriting the future and by shuffling labels. Bootstrap intervals resample whole matches or innings.

Why Holm correction, and what did you find?

I ran 20 tests, so the chance of a false positive grows; Holm controls the family-wise error rate and is less conservative than Bonferroni. The toss had no detectable effect (48.1% in ODI, 51.5% in T20), batting first no significant edge, while the ODI scoring trend (about 3 runs a year) was significant. I report effect sizes and intervals, not only p-values.

What metrics did you use and why?
Accuracy, precision, recall, F1 and ROC-AUC, plus log loss, Brier score and calibration curves, because win probabilities should be well calibrated, not only right on average. For regression: MAE, RMSE and R2.

How do you explain the models?
Coefficients and permutation importance, SHAP values and ablation (remove a feature group and measure the loss). The report states which features matter and warns about small-sample noise.

Backend, frontend and deployment questions

Why the Streamlit shim instead of rewriting the pages?
It preserved ten working, tested analytics pages and avoided a risky rewrite. Page code runs unchanged; the shim records each st.* call into a JSON tree for React. The trade-off is that React renders a generic tree, so very custom UI needs a new node type, which I added for the match centre.

How does a click in React reach Python and back?
React posts the widget state to /api/page/{slug}. The server runs the page against that state (cached by competition, page, state and edition), the shim returns the JSON tree, and React renders it. Table row clicks set a session value that the match page reads as its default selection.

How did you theme every tournament?
Each edition has a palette taken from its logo. CSS variables restyle the app, and a recolour function maps Plotly's base colours to the edition's palette. Selecting an edition also sets sensible defaults on every page.

How did you design the match centre?
One payload per match: header, scoreboard lines, scorecards, ball chips, partnerships and stats, all from the stored data. It hides unsupported stats rather than showing zeros, shows the winner prominently without hiding the loser, and handles abandoned matches, walkovers, super overs and Tests.

How do you handle flags correctly?
A canonical team-name map with aliases (USA, UAE and similar) points to local SVG files, so it works offline and on every OS. Teams without a national flag, such as West Indies, show a neutral badge so a wrong flag is never displayed.

How did you deploy, and what went wrong?
A FastAPI function plus static React on Vercel. The bundle was 590 MB against a 500 MB limit, then routes returned 404, DuckDB would not open, and pandas differences broke pages. I compacted databases, moved to a root entry point, and pinned every library. Then I compared local and live across 305 page runs to prove parity.

How do you test it?
340 automated tests: parsers, quality rules, reconciliation of innings against scorecards, ML and leakage checks, the dashboard pages under real Streamlit, the React backend and match centre, and the competitions data. For deployment I used a live-versus-local parity script.

What did you get wrong, and what did you learn?
I once reported that every page loaded after checking only HTTP status codes, but errors were returned as page content. Comparing the content structure found the real failures. The lesson: verify outputs, not only that a request succeeded, and say plainly when a check was incomplete.

What would you improve next?

Add pitch, weather and day/night data (the main modelling gap), CI that runs tests and the parity check on every push, a managed database for larger data, women's cricket and more competitions, and live data. I would also add caching on the CDN for popular pages and an incremental pipeline instead of full rebuilds.

10. Quick reference

Numbers to remember

Topic	Number
World Cup matches / editions	677 matches, 16 editions (ODI 2003-2023, T20 2007-2026)
Champions Trophy / Test Championship	112 matches (7 editions) / 245 Tests (4 cycles)
Deliveries	193,779 (World Cups), 47,032 (Champions Trophy), 439,542 (Test Championship)
Ball-by-ball coverage (World Cups)	488 of 652 played matches (75%)
Data-quality score	99.7 / 100
Pipeline / dbt / tests	20 steps / 31 SQL models / 340 tests
Winner model	Elo 66.0%, logistic 68.6%, best 69.9% (n = 465); coin 48.8%
ODI score projection	baseline MAE 47.0 (R2 0.24) to MAE 33.0 (R2 0.62)
Toss effect	ODI 48.1%, T20 51.5%: not significant
Vercel bundle	475 MB of 500 MB; 305 page runs matched local
World Cup 2027	57 fixtures, 12 venues, 3 hosts, 10 confirmed teams + 4 qualifier places

Run it

Task	Command
Install	pip install -r requirements.txt; cd web; npm install; npm run build
Run the dashboard	python scripts/run_react_dashboard.py (http://localhost:8000)
Run the full pipeline	python scripts/run_lakehouse_pipeline.py
Add competitions / 2027	python scripts/run_competitions.py / python scripts/run_competitions.py --wc2027
Tests	python -m pytest tests -q
Deploy bundle	python scripts/build_vercel_bundle.py build/vercel; vercel deploy --prod

Honest limits to mention before you are asked

- Only about 640 decided matches, so model gains are small and intervals are wide.
- 75% ball-by-ball coverage for World Cups; Champions Trophy 2002 has no ball-by-ball.
- Pitch, weather and day/night are not modelled.
- 2025-27 Test Championship has no champion yet; World Cup 2027 holds only published information and four teams are still to qualify.
- ML and the World Cup statistics page are trained on World Cup data only.
- Venue map coordinates and capacities are approximate; the live databases are compacted copies (same rows).

Links

Live dashboard: https://icccricketanaylsis.vercel.app. Documentation: docs/architecture.md, docs/competitions.md, docs/data_dictionary.md, reports/machine_learning_report.md, reports/data_quality_report.md.